МИНОБРНАУКИ РОССИИ
федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технологический университет «СТАНКИН» (ФГАОУ ВО «МГТУ «СТАНКИН»)

Институт информационных технологий

Кафедра управления и информатики в технических системах

ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ № 1 ПО ДИСЦИПЛИНЕ

«Прикладное программирование»

СТУДЕНТА *2* КУРСА *бакалавриата* ГРУППЫ *ИДБ-25-12*
(уровень профессионального образования)

Богданов Даниил Алексеевич

НА ТЕМУ

ООП в Python. Обработка исключительных ситуаций.
Форматы XML и JSON

Направление: 09.03.03 Прикладная информатика
Профиль подготовки: Управление данными

Отчет сдан «_____»_____20___г.

Оценка _____

Преподаватель Гетманов С. В. _____
(Ф.И.О., должность, степень, звание.) *(подпись)*

МОСКВА 2026

СОДЕРЖАНИЕ

1 Цель работы..... 3
2 Задание..... 3
3 Ход работы..... 3
3.1 Описание предметной области..... 3
3.2 Диаграмма классов..... 4
3.3 Описание классов..... 6
3.4 Обработка исключений..... 7
3.5 Структура хранения данных в форматах JSON и XML..... 8
3.6 Реализация CRUD, чтения и записи файлов..... 9
3.7 Пользовательский интерфейс..... 10
3.8 Работа с системой контроля версий..... 12
4 Вывод..... 12
Приложение А. Листинг программного кода..... 13
Файл models.py..... 13
Файл exceptions.py..... 20
Файл repository.py..... 21
Файл service.py..... 22
Файл storage.py..... 26
Файл sample_data.py..... 28
Файл cli.py..... 30
Файл __init__.py..... 31
Файл __main__.py..... 31

1 Цель работы

Изучить объектно-ориентированное программирование на языке Python на примере предметной области онлайн-кинотеатра, научиться проектировать классы и связи между ними, обрабатывать встроенные и собственные исключения, а также разрабатывать структуры хранения данных в форматах XML и JSON и выполнять чтение и запись этих форматов.

2 Задание

Вариант 4: написать классы для предметной области онлайн-кинотеатра.

Требуется:

1. спроектировать диаграмму классов для выбранной предметной области и написать по ней код на Python в объектно-ориентированном стиле;
2. обработать встроенные и собственные исключения так, чтобы при ошибках программа не завершалась, и актуализировать диаграмму классов;
3. спроектировать структуру хранения данных в форматах XML и JSON;
4. реализовать считывание из файла и запись в файл по разработанной структуре в форматах XML и JSON;
5. подготовить отчёт о выполненной работе и подготовиться к защите.

3 Ход работы

3.1 Описание предметной области

Программа предназначена для моделирования основных процессов онлайн-кинотеатра. В ней хранятся пользователи, фильмы, жанры, тарифные планы, подписки, отзывы, история просмотров, списки избранного, платежи, промокоды и устройства, с которых пользователь обращается к сервису.

Объекты связаны между собой через идентификаторы (id). Например, фильм содержит список id жанров; подписка содержит id пользователя и тарифного плана; отзыв содержит id пользователя и фильма; платеж связан с пользователем и подпиской. Это позволяет хранить связанные данные раздельно и проверять корректность ссылок.

Для всех 11 типов объектов на уровне сервисного слоя реализованы базовые операции CRUD: добавление (Create), получение (Read), изменение (Update) и удаление (Delete). Программа также позволяет выполнять поиск фильмов и сохранять полный набор данных в файлы JSON и XML.

Исходный код расположен в пакете online_cinema. В корне проекта находятся папки data, diagrams, docs и prototypes, а также README.md с краткой инструкцией по запуску.

Модуль	Содержимое
models.py	11 классов предметной области, перечисления статусов и класс CinemaData.
exceptions.py	Собственные исключения приложения.
repository.py	Класс InMemoryRepository для хранения объектов и базовых операций CRUD.
service.py	Класс CinemaService: бизнес-правила, проверка связей и работа с объектами.
storage.py	Классы StorageAdapter, JsonStorage и XmlStorage для чтения и записи файлов.
sample_data.py	Создание демонстрационного набора данных.
cli.py	Консольное меню и обработка команд пользователя.
__main__.py	Точка запуска приложения.

3.2 Диаграмма классов

Диаграмма классов подготовлена в PlantUML (исходный файл class_diagram.puml) как средство, аналогичное draw.io. На рисунке 1 представлены 11 классов предметной области, их основные атрибуты и связи. Исходник диаграммы можно изменять, а PNG-файл используется для просмотра и включения в отчёт.

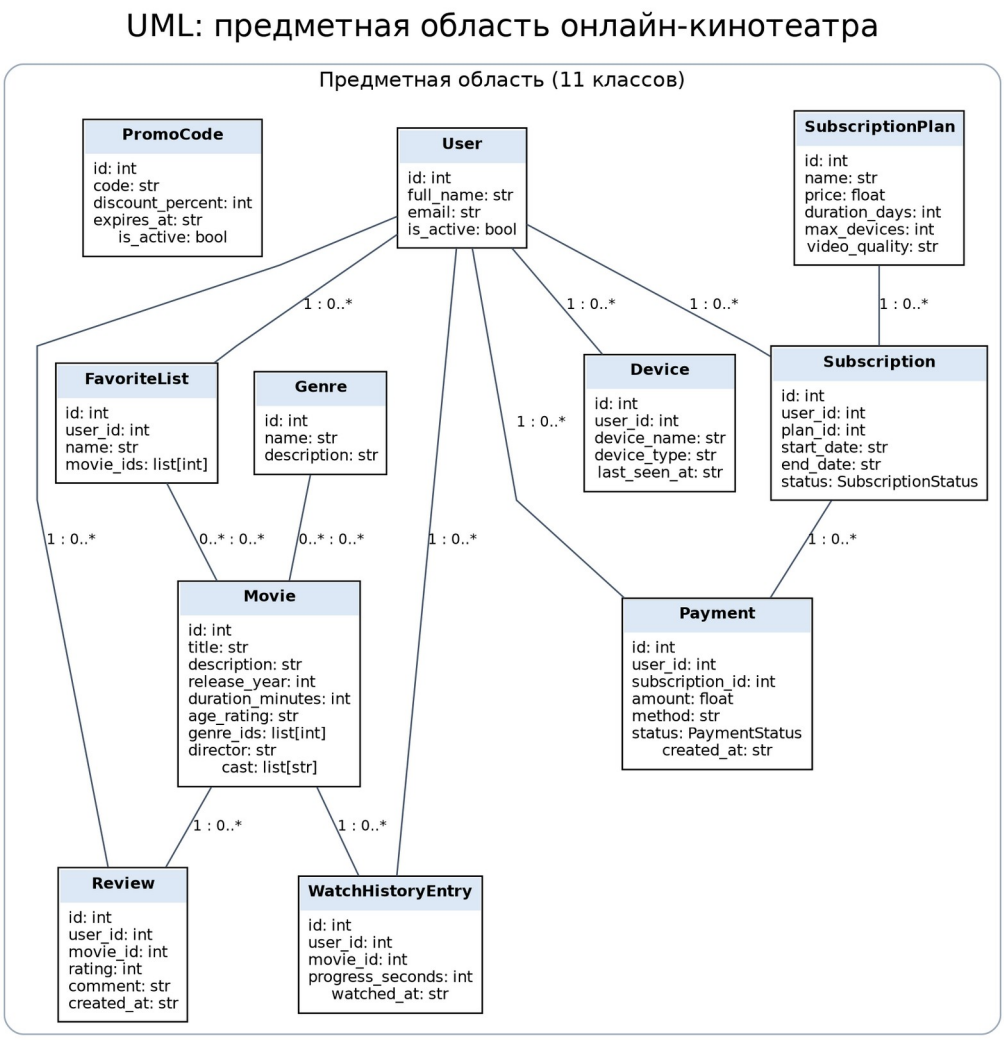


Рисунок 1 — Диаграмма классов предметной области онлайн-кинотеатра

На диаграмме показаны связи «один ко многим» между пользователем и подписками, отзывами, историей просмотров и устройствами; между тарифом и подписками; между подпиской и платежами. Связь фильма с жанрами и списками избранного может быть множественной. В коде эти связи представлены полями с id.

3.3 Описание классов

Для описания предметной области создано 11 основных классов. Дополнительно в программе используются класс CinemaData для объединения коллекций, перечисления статусов, классы хранения, сервисный класс и классы исключений.

Класс	Назначение
User	Пользователь: имя, электронная почта, состояние учётной записи.
Genre	Жанр фильма: название и описание.
Movie	Фильм: название, описание, год выпуска, длительность, возрастной рейтинг, жанры, режиссёр и актёрский состав.
SubscriptionPlan	Тариф: стоимость, срок действия, максимальное число устройств и качество видео.
Subscription	Подписка пользователя: тариф, даты начала и окончания, статус.
Review	Отзыв пользователя о фильме: оценка, комментарий и дата создания.
WatchHistoryEntry	Запись истории просмотра: пользователь, фильм, прогресс и время просмотра.
FavoriteList	Список избранного: владелец, название и список id фильмов.
Payment	Платёж: пользователь, подписка, сумма, способ и статус оплаты.
PromoCode	Промокод: код, размер скидки, срок действия и признак активности.
Device	Устройство пользователя: название, тип и время последней активности.

Принципы объектно-ориентированного программирования в программе:

- инкапсуляция — данные и проверки находятся в соответствующих классах. После создания объекта метод __post_init__ проверяет значения его полей;
- наследование — классы JsonStorage и XmlStorage реализуют общий базовый интерфейс StorageAdapter; собственные исключения наследуются от CinemaError;
- полиморфизм — оба класса хранилищ имеют одинаковые методы save() и load(), но сохраняют и читают данные в разных форматах.

Классы предметной области объявлены с помощью dataclass. Это позволяет автоматически создавать конструкторы и хранить наборы полей без большого количества однотипного кода. Для функций и методов используются аннотации типов. Имена переменных и функций оформлены в стиле snake_case, классов — в стиле CapWords; код разделён на модули.

3.4 Обработка исключений

Собственные исключения объявлены в модуле exceptions.py. Общий класс CinemaError является родителем остальных исключений, поэтому ошибки предметной области можно перехватывать одним блоком except.

Исключение	Когда возникает
CinemaError	Базовое исключение приложения.
ValidationError	Поле пустое или имеет неверный тип/значение; рейтинг, дата, сумма или статус не соответствуют правилам.
EntityNotFoundError	Объект с указанным id отсутствует в репозитории.
DuplicateEntityError	Добавляется объект с занятым id, email, названием жанра или промокодом.
BusinessRuleError	Операция нарушает правило связей, например удаление используемого фильма или смена владельца подписки при наличии платежа.
StorageError	Не удалось прочитать, разобрать или записать файл JSON/XML.

Также обрабатываются встроенные исключения Python: ValueError и TypeError при преобразовании и проверке введённых значений; KeyError при обращении к отсутствующему ключу; OSError при операциях с файлами; json.JSONDecodeError при ошибке разбора JSON; xml.etree.ElementTree.ParseError при ошибке разбора XML; EOFError и KeyboardInterrupt при прерывании ввода.

В консольном интерфейсе ошибки перехватываются в основном цикле меню. Сообщение выводится пользователю, после чего программа возвращается к меню. При чтении файла ошибки преобразуются в StorageError. Это позволяет не завершать программу из-за ожидаемых ошибок ввода или некорректного файла.

Пример обработки ошибки пользовательского ввода в cli.py:

```
try:
    choice = input("Выберите действие: ").strip()
    if choice == "1":
        _print_movies(service)
    elif choice == "0":
        break
except (CinemaError, ValueError, TypeError) as exc:
    print(f"Ошибка: {exc}")
except (EOFError, KeyboardInterrupt):
    print("\nРабота завершена пользователем.")
```

3.5 Структура хранения данных в форматах JSON и XML

Все данные онлайн-кинотеатра сохраняются одним набором. В JSON корневым элементом является объект, содержащий коллекции users, genres, movies, plans, subscriptions, reviews, watch_history, favorite_lists, payments, promo_codes и devices. Каждая коллекция представлена массивом объектов. Связи между объектами записываются через id, даты и время — строками в формате ISO.

Фрагмент файла data/online_cinema.json:

```
{
    "users": [
        {
            "id": 1,
            "full_name": "Иван Петров",
            "email": "ivan@example.com",
            "is_active": true
        }
    ],
    "genres": [
        {"id": 1, "name": "Фантастика", "description": "..."}
    ],
    "movies": [
        {
            "id": 1,
            "title": "Последний сигнал",
            "release_year": 2024,
            "duration_minutes": 118,
            "age_rating": "16+",
            "genre_ids": [1, 2]
        }
    ]
}
```

В XML используется корневой элемент online_cinema с атрибутом schema_version. Внутри него расположены элементы коллекций, а каждый объект представлен элементом item. Поля объекта записываются вложенными тегами. Списки, например genre_ids и cast, также представлены вложенными элементами item.

Фрагмент файла data/online_cinema.xml:

```
<?xml version='1.0' encoding='utf-8'?>
<online_cinema schema_version="1.0">
    <users>
        <item>
            <id>1</id>
            <full_name>Иван Петров</full_name>
            <email>ivan@example.com</email>
            <is_active>True</is_active>
```

```
        </item>
    </users>
    <genres>
        <item>
            <id>1</id>
            <name>Фантастика</name>
        </item>
    </genres>
</online_cinema>
```

3.6 Реализация CRUD, чтения и записи файлов

Класс InMemoryRepository хранит объекты одного типа в словаре, где ключом является id. Он предоставляет методы add(), get(), get_all(), update() и delete(). Класс CinemaService создаёт отдельный репозиторий для каждого из 11 типов объектов и выполняет проверки перед изменением данных.

Операция	Метод	Назначение
Create	create(entity)	Добавляет новый объект после проверки id, уникальности и связей.
Read	get(type, id), get_all(type)	Возвращает один объект или список объектов выбранного типа.
Update	update(type, id, **changes)	Создаёт обновлённую копию объекта и повторно проверяет её поля.
Delete	delete(type, id)	Удаляет объект, если на него не ссылаются другие сущности.

Дополнительно CinemaService проверяет уникальность электронной почты пользователя, названия жанра и промокода. При создании фильма проверяется наличие указанных жанров; при создании подписки — пользователя и тарифа; при создании платежа — пользователя и подписки. Удаление объектов, которые используются другими записями, запрещается.

Для переноса данных между сервисом и файловым хранилищем используется CinemaData. Метод export_data() собирает все коллекции, а import_data() восстанавливает объекты и повторно проверяет связи. JsonStorage использует модуль json, XmlStorage — xml.etree.ElementTree. После чтения из файла данные преобразуются обратно в экземпляры классов.

Пример сохранения и загрузки данных:

```
data = service.export_data()
JsonStorage().save(data, "data/online_cinema.json")
XmlStorage().save(data, "data/online_cinema.xml")
service.import_data(JsonStorage().load("data/online_cinema.json"))
```

Работоспособность файлового обмена проверяется на демонстрационном наборе: данные сохраняются в JSON и XML, затем загружаются обратно. При чтении файлов выполняется проверка структуры и значений; ошибки разбора преобразуются в понятное исключение StorageError.

3.7 Пользовательский интерфейс

Для работы с программой предусмотрено консольное меню. Оно позволяет просматривать каталог фильмов, выполнять поиск по названию, описанию или режиссёру, добавлять фильм, изменять его описание и удалять фильм. Также предусмотрены команды сохранения и загрузки набора данных в форматах JSON и XML.

Основные пункты консольного меню:

```
=== LUMEN | Онлайн-кинотеатр ===
1. Показать каталог
2. Найти фильм
3. Добавить фильм
4. Изменить описание фильма
5. Удалить фильм
6. Сохранить данные в JSON
7. Сохранить данные в XML
8. Загрузить данные из JSON
9. Загрузить данные из XML
0. Выход
```

Дополнительно подготовлен статический прототип интерфейса в HTML, CSS и JavaScript. В нём представлены главная страница онлайн-кинотеатра и каталог фильмов. Прототип демонстрирует внешний вид и навигацию, но не подключён к Python-приложению и не воспроизводит реальное видео.

Главная страница прототипа:

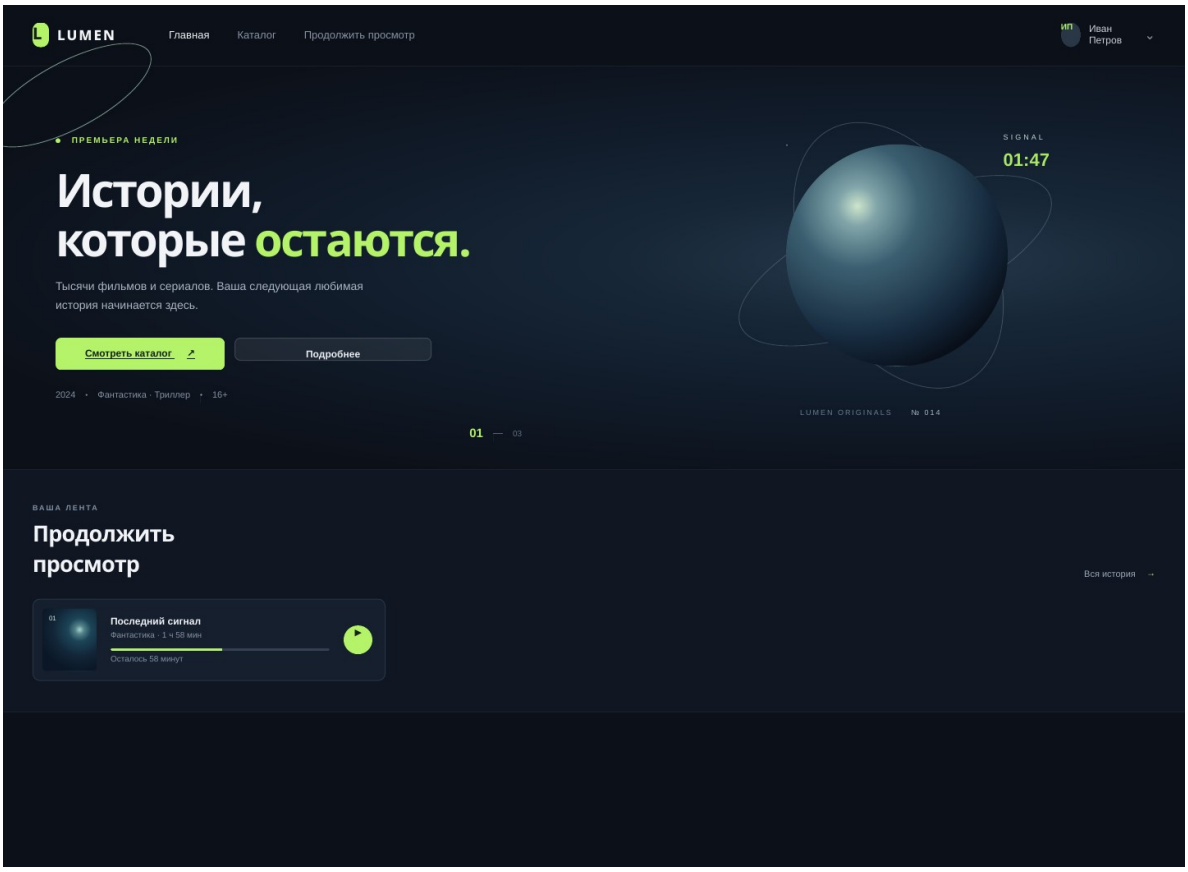


Рисунок 2 — Главная страница прототипа LUMEN

Страница каталога фильмов:

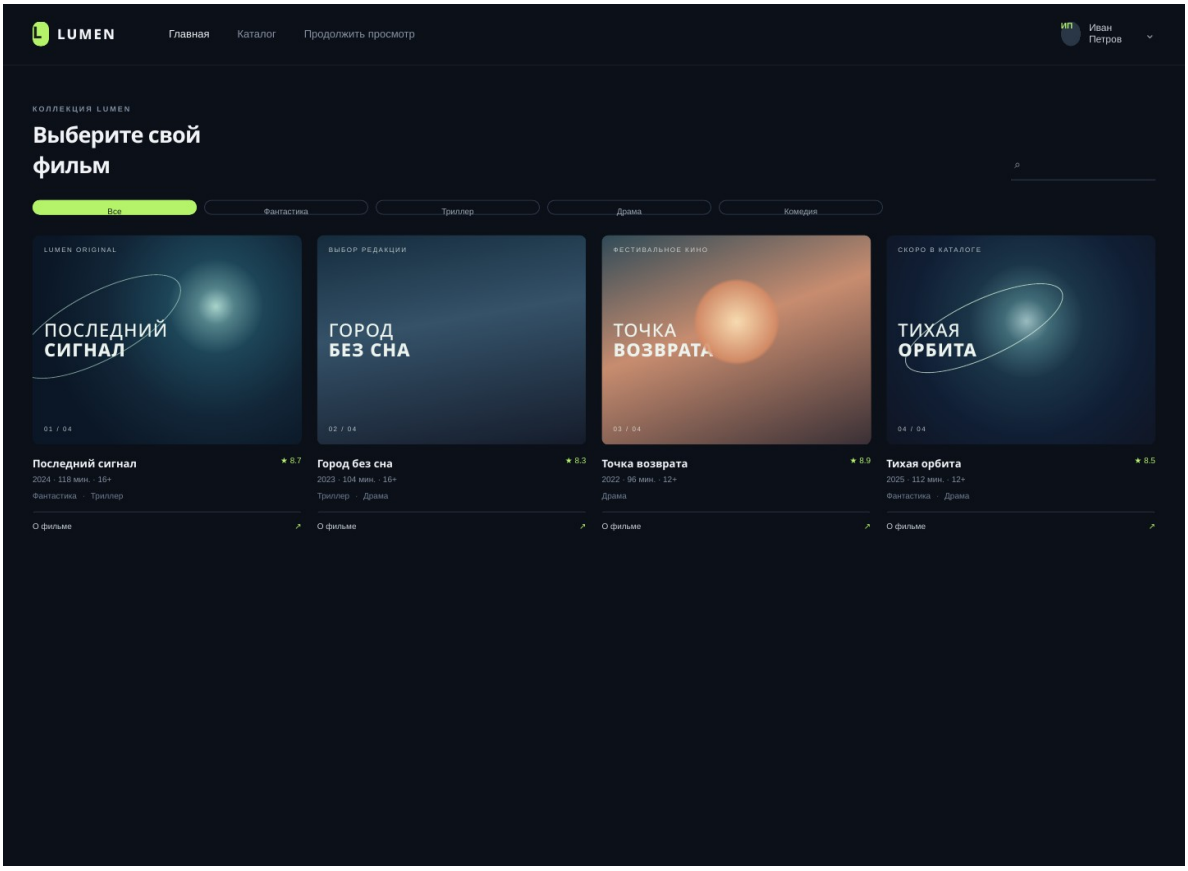


Рисунок 3 — Каталог фильмов прототипа LUMEN

3.8 Работа с системой контроля версий

Для контроля версий проекта используется Git. Исходный код, демонстрационные данные, UML-диаграмма, прототип интерфейса и отчёт размещены в одном репозитории. В корне проекта находится README.md с описанием структуры и командой запуска.

Репозиторий проекта размещён на GitHub по следующему адресу:

https://github.com/palesiemens/lr1-online-cinema

4 Вывод

В ходе лабораторной работы для предметной области «Онлайн-кинотеатр» спроектирована UML-диаграмма и разработана программа на Python в объектно-ориентированном стиле. Созданы 11 классов предметной области, реализованы операции CRUD, проверка значений и связей между объектами. Разработана иерархия собственных исключений, а ожидаемые ошибки ввода и работы с файлами обрабатываются без аварийного завершения консольного меню.

Для хранения данных спроектированы структуры JSON и XML, реализованы сериализация и десериализация. Подготовлены консольный интерфейс и статический HTML-прототип. Программа использует стандартную библиотеку Python, поэтому для её запуска не требуется устанавливать дополнительные пакеты или создавать виртуальное окружение.

Приложение А. Листинг программного кода

Файл models.py

```
"""Модели предметной области онлайн-кинотеатра."""


from __future__ import annotations


from dataclasses import asdict, dataclass, field, fields
from datetime import date, datetime
from enum import Enum
import math
import re
from typing import Any, TypeVar, get_args, get_origin, get_type_hints


from .exceptions import ValidationError



class SubscriptionStatus(str, Enum):
    """Состояние пользовательской подписки."""


    ACTIVE = "active"
    EXPIRED = "expired"
    CANCELLED = "cancelled"



class PaymentStatus(str, Enum):
    """Состояние платежа."""


    PENDING = "pending"
    PAID = "paid"
    FAILED = "failed"
    REFUNDED = "refunded"



def _check_integer(value: int, field_name: str, minimum: int | None = None) -> None:
    """Проверяет, что значение является целым числом и не ниже минимума."""
    if type(value) is not int:
        raise ValidationError(f"Поле «{field_name}» должно быть целым числом.")
    if minimum is not None and value < minimum:
        raise ValidationError(
            f"Поле «{field_name}» должно быть не меньше {minimum}."
        )



def _check_positive(value: int, field_name: str) -> None:
    _check_integer(value, field_name, minimum=1)



def _check_amount(value: int | float, field_name: str) -> None:
    """Проверяет конечное неотрицательное числовое значение."""
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise ValidationError(f"Поле «{field_name}» должно быть числом.")
    if (isinstance(value, float) and not math.isfinite(value)) or value < 0:
        raise ValidationError(
            f"Поле «{field_name}» должно быть конечным и неотрицательным."
        )



def _check_bool(value: bool, field_name: str) -> None:
    if type(value) is not bool:
        raise ValidationError(f"Поле «{field_name}» должно быть логическим.")
```

```
def _check_text(value: str, field_name: str) -> None:
    if not isinstance(value, str) or not value.strip():
        raise ValidationError(f"Поле «{field_name}» не должно быть пустым.")



def _check_iso_date(value: str, field_name: str) -> None:
    try:
        date.fromisoformat(value)
    except (TypeError, ValueError) as exc:
        raise ValidationError(
            f"Поле «{field_name}» должно содержать дату в формате ГГГГ-ММ-ДД."
        ) from exc



def _check_iso_datetime(value: str, field_name: str) -> None:
    try:
        datetime.fromisoformat(value)
    except (TypeError, ValueError) as exc:
        raise ValidationError(
            f"Поле «{field_name}» должно содержать дату и время ISO 8601."
        ) from exc



@dataclass(slots=True)
class Genre:
    """Жанр фильма."""


    id: int
    name: str
    description: str = ""


    def __post_init__(self) -> None:
        _check_positive(self.id, "id жанра")
        _check_text(self.name, "название жанра")



@dataclass(slots=True)
class Movie:
    """Карточка фильма в каталоге."""


    id: int
    title: str
    description: str
    release_year: int
    duration_minutes: int
    age_rating: str
    genre_ids: list[int] = field(default_factory=list)
    director: str = "Не указан"
    cast: list[str] = field(default_factory=list)


    def __post_init__(self) -> None:
        _check_positive(self.id, "id фильма")
        _check_text(self.title, "название фильма")
        _check_text(self.description, "описание фильма")
        _check_integer(self.release_year, "год выпуска")
        if not 1888 <= self.release_year <= 2100:
            raise ValidationError(
                "Год выпуска должен находиться в диапазоне 1888-2100."
            )
        _check_positive(self.duration_minutes, "длительность фильма")
        if not isinstance(self.age_rating, str) or self.age_rating not in {
            "0+", "6+", "12+", "16+", "18+"
        }:
            raise ValidationError(
                "Возрастной рейтинг должен быть одним из: 0+, 6+, 12+, 16+, 18+."
            )
        if not isinstance(self.genre_ids, list) or any(
            type(value) is not int or value <= 0 for value in self.genre_ids
        ):
            raise ValidationError(
                "Идентификаторы жанров должны быть положительными целыми числами."
```

```
            )
        if len(self.genre_ids) != len(set(self.genre_ids)):
            raise ValidationError("Список жанров фильма не должен содержать повторов.")
        if not isinstance(self.cast, list) or any(
            not isinstance(name, str) or not name.strip() for name in self.cast
        ):
            raise ValidationError("Список актеров должен содержать непустые строки.")


@dataclass(slots=True)
class User:
    """Учетная запись зрителя."""

    id: int
    full_name: str
    email: str
    is_active: bool = True

    def __post_init__(self) -> None:
        _check_positive(self.id, "id пользователя")
        _check_text(self.full_name, "имя пользователя")
        _check_text(self.email, "электронная почта")
        self.email = self.email.strip().lower()
        if not re.fullmatch(r"[^\s@]+@[^\s@]+\.[^\s@]+", self.email):
            raise ValidationError("Указан некорректный адрес электронной почты.")
        _check_bool(self.is_active, "состояние учетной записи")


@dataclass(slots=True)
class SubscriptionPlan:
    """Тарифный план онлайн-кинотеатра."""

    id: int
    name: str
    price: float
    duration_days: int
    max_devices: int
    video_quality: str = "Full HD"

    def __post_init__(self) -> None:
        _check_positive(self.id, "id тарифа")
        _check_text(self.name, "название тарифа")
        _check_amount(self.price, "стоимость тарифа")
        _check_positive(self.duration_days, "срок действия тарифа")
        _check_positive(self.max_devices, "количество устройств")
        if not isinstance(self.video_quality, str) or self.video_quality not in {
            "HD", "Full HD", "4K"
        }:
            raise ValidationError("Качество видео должно быть HD, Full HD или 4K.")


@dataclass(slots=True)
class Subscription:
    """Подписка конкретного пользователя."""

    id: int
    user_id: int
    plan_id: int
    start_date: str
    end_date: str
    status: SubscriptionStatus = SubscriptionStatus.ACTIVE

    def __post_init__(self) -> None:
        _check_positive(self.id, "id подписки")
        _check_positive(self.user_id, "id пользователя подписки")
        _check_positive(self.plan_id, "id тарифа подписки")
        _check_iso_date(self.start_date, "дата начала подписки")
        _check_iso_date(self.end_date, "дата окончания подписки")
        if date.fromisoformat(self.end_date) <= date.fromisoformat(self.start_date):
            raise ValidationError(
                "Дата окончания подписки должна быть позже даты начала."
```

```
            )
        try:
            self.status = SubscriptionStatus(self.status)
        except ValueError as exc:
            raise ValidationError("Неизвестный статус подписки.") from exc



@dataclass(slots=True)
class Review:
    """Оценка фильма, оставленная пользователем."""

    id: int
    user_id: int
    movie_id: int
    rating: int
    comment: str = ""
    created_at: str = field(
        default_factory=lambda: datetime.now().isoformat(timespec="seconds")
    )

    def __post_init__(self) -> None:
        _check_positive(self.id, "id отзыва")
        _check_positive(self.user_id, "id пользователя отзыва")
        _check_positive(self.movie_id, "id фильма отзыва")
        _check_integer(self.rating, "оценка фильма")
        if not 1 <= self.rating <= 10:
            raise ValidationError("Оценка фильма должна быть целым числом от 1 до 10.")
        if not isinstance(self.comment, str):
            raise ValidationError("Комментарий должен быть текстовой строкой.")
        _check_iso_datetime(self.created_at, "дата создания отзыва")



@dataclass(slots=True)
class WatchHistoryEntry:
    """Запись истории просмотра."""

    id: int
    user_id: int
    movie_id: int
    progress_seconds: int = 0
    watched_at: str = field(
        default_factory=lambda: datetime.now().isoformat(timespec="seconds")
    )

    def __post_init__(self) -> None:
        _check_positive(self.id, "id записи истории")
        _check_positive(self.user_id, "id пользователя истории")
        _check_positive(self.movie_id, "id фильма истории")
        _check_integer(self.progress_seconds, "прогресс просмотра", minimum=0)
        _check_iso_datetime(self.watched_at, "дата просмотра")



@dataclass(slots=True)
class FavoriteList:
    """Именованный список избранных фильмов пользователя."""

    id: int
    user_id: int
    name: str = "Избранное"
    movie_ids: list[int] = field(default_factory=list)

    def __post_init__(self) -> None:
        _check_positive(self.id, "id списка избранного")
        _check_positive(self.user_id, "id пользователя списка")
        _check_text(self.name, "название списка")
        if not isinstance(self.movie_ids, list) or any(
            type(value) is not int or value <= 0 for value in self.movie_ids
        ):
            raise ValidationError(
                "Идентификаторы фильмов должны быть положительными целыми числами."
            )
```

```
        if len(self.movie_ids) != len(set(self.movie_ids)):
            raise ValidationError(
                "В списке избранного не должно быть повторяющихся фильмов."
            )



@dataclass(slots=True)
class Payment:
    """Платеж за подписку."""

    id: int
    user_id: int
    subscription_id: int
    amount: float
    method: str
    status: PaymentStatus = PaymentStatus.PENDING
    created_at: str = field(
        default_factory=lambda: datetime.now().isoformat(timespec="seconds")
    )

    def __post_init__(self) -> None:
        _check_positive(self.id, "id платежа")
        _check_positive(self.user_id, "id пользователя платежа")
        _check_positive(self.subscription_id, "id подписки платежа")
        _check_amount(self.amount, "сумма платежа")
        _check_text(self.method, "способ оплаты")
        _check_iso_datetime(self.created_at, "дата платежа")
        try:
            self.status = PaymentStatus(self.status)
        except ValueError as exc:
            raise ValidationError("Неизвестный статус платежа.") from exc



@dataclass(slots=True)
class PromoCode:
    """Промокод для предоставления скидки."""

    id: int
    code: str
    discount_percent: int
    expires_at: str
    is_active: bool = True

    def __post_init__(self) -> None:
        _check_positive(self.id, "id промокода")
        _check_text(self.code, "код промокода")
        self.code = self.code.strip().upper()
        _check_bool(self.is_active, "активность промокода")
        _check_integer(self.discount_percent, "размер скидки")
        if not 1 <= self.discount_percent <= 100:
            raise ValidationError("Размер скидки должен быть от 1 до 100 процентов.")
        _check_iso_date(self.expires_at, "срок действия промокода")



@dataclass(slots=True)
class Device:
    """Устройство, с которого пользователь входит в сервис."""

    id: int
    user_id: int
    device_name: str
    device_type: str
    last_seen_at: str = field(
        default_factory=lambda: datetime.now().isoformat(timespec="seconds")
    )

    def __post_init__(self) -> None:
        _check_positive(self.id, "id устройства")
        _check_positive(self.user_id, "id пользователя устройства")
        _check_text(self.device_name, "название устройства")
        if not isinstance(self.device_type, str) or self.device_type not in {
```

```
            "smart_tv", "computer", "smartphone", "tablet", "other"
        }:
            raise ValidationError("Неизвестный тип устройства.")
        _check_iso_datetime(self.last_seen_at, "время последней активности")



T = TypeVar("T")



def _serialise(value: Any) -> Any:
    """Преобразует вложенные dataclass и Enum в типы JSON/XML."""
    if isinstance(value, Enum):
        return value.value
    if isinstance(value, list):
        return [_serialise(item) for item in value]
    if isinstance(value, dict):
        return {key: _serialise(item) for key, item in value.items()}
    return value



def _coerce_value(value: Any, expected_type: Any) -> Any:
    """Восстанавливает тип поля после чтения JSON или XML."""
    origin = get_origin(expected_type)
    args = get_args(expected_type)


    if origin is list:
        item_type = args[0] if args else Any
        if value in (None, ""):
            return []
        if not isinstance(value, list):
            raise ValueError("Ожидался список значений.")
        return [_coerce_value(item, item_type) for item in value]
    if expected_type is Any:
        return value
    if isinstance(expected_type, type) and issubclass(expected_type, Enum):
        return expected_type(value)
    if expected_type is bool:
        if isinstance(value, bool):
            return value
        normalized = str(value).strip().lower()
        if normalized in {"true", "1", "yes"}:
            return True
        if normalized in {"false", "0", "no"}:
            return False
        raise ValueError(f"Некорректное логическое значение: {value!r}.")
    if expected_type is int:
        if type(value) is int:
            return value
        if isinstance(value, str):
            return int(value)
        raise ValueError(f"Ожидалось целое число, получено {value!r}.")
    if expected_type is float:
        if isinstance(value, bool) or not isinstance(value, (int, float, str)):
            raise ValueError(f"Ожидалось число, получено {value!r}.")
        return float(value)
    if expected_type is str:
        if not isinstance(value, str):
            raise ValueError(f"Ожидалась строка, получено {value!r}.")
        return value
    return value



@dataclass(slots=True)
class CinemaData:
    """Полный набор данных, переносимый между хранилищами."""


    users: list[User] = field(default_factory=list)
    genres: list[Genre] = field(default_factory=list)
    movies: list[Movie] = field(default_factory=list)
    plans: list[SubscriptionPlan] = field(default_factory=list)
    subscriptions: list[Subscription] = field(default_factory=list)
    reviews: list[Review] = field(default_factory=list)
    watch_history: list[WatchHistoryEntry] = field(default_factory=list)
```

```
favorite_lists: list[FavoriteList] = field(default_factory=list)
payments: list[Payment] = field(default_factory=list)
promo_codes: list[PromoCode] = field(default_factory=list)
devices: list[Device] = field(default_factory=list)


def to_dict(self) -> dict[str, Any]:
    """Возвращает сериализуемый словарь со всеми коллекциями."""
    return _serialise(asdict(self))


@classmethod
def from_dict(cls, payload: dict[str, Any]) -> CinemaData:
    """Создает типизированные объекты из словаря, прочитанного из файла."""
    model_by_collection: dict[str, type[Any]] = {
        "users": User,
        "genres": Genre,
        "movies": Movie,
        "plans": SubscriptionPlan,
        "subscriptions": Subscription,
        "reviews": Review,
        "watch_history": WatchHistoryEntry,
        "favorite_lists": FavoriteList,
        "payments": Payment,
        "promo_codes": PromoCode,
        "devices": Device,
    }
    result: dict[str, Any] = {}
    for collection_name, model_type in model_by_collection.items():
        objects: list[Any] = []
        for raw_object in payload.get(collection_name, []):
            hints = get_type_hints(model_type)
            values = {
                item.name: _coerce_value(raw_object[item.name], hints[item.name])
                for item in fields(model_type)
                if item.name in raw_object
            }
            objects.append(model_type(**values))
        result[collection_name] = objects
    return cls(**result)
```

Файл exceptions.py

```
"""Пользовательские исключения приложения онлайн-кинотеатра."""



class CinemaError(Exception):
    """Базовое исключение предметной области онлайн-кинотеатра."""



class ValidationError(CinemaError):
    """Ошибка проверки значений объекта или входных данных."""



class EntityNotFoundError(CinemaError):
    """Запрашиваемая сущность отсутствует в хранилище."""



class DuplicateEntityError(CinemaError):
    """Попытка добавить сущность с уже используемым идентификатором или ключом."""



class BusinessRuleError(CinemaError):
    """Операция нарушает бизнес-правило или целостность связей."""



class StorageError(CinemaError):
    """Ошибка чтения, записи или разбора файла данных."""
```

Файл repository.py

```
"""Универсальное хранилище объектов в оперативной памяти."""

from __future__ import annotations

from typing import Generic, TypeVar

from .exceptions import DuplicateEntityError, EntityNotFoundError

T = TypeVar("T")



class InMemoryRepository(Generic[T]):
    """Реализует базовые операции CRUD для объектов с целочисленным id."""

    def __init__(self, entity_name: str) -> None:
        self._entity_name = entity_name
        self._items: dict[int, T] = {}

    def add(self, entity: T) -> T:
        entity_id = int(getattr(entity, "id"))
        if entity_id in self._items:
            raise DuplicateEntityError(
                f"{self._entity_name} с id={entity_id} уже существует."
            )
        self._items[entity_id] = entity
        return entity

    def get(self, entity_id: int) -> T:
        try:
            return self._items[entity_id]
        except KeyError as exc:
            raise EntityNotFoundError(
                f"{self._entity_name} с id={entity_id} не найден(а)."
            ) from exc

    def get_all(self) -> list[T]:
        return [self._items[key] for key in sorted(self._items)]

    def update(self, entity_id: int, entity: T) -> T:
        if entity_id not in self._items:
            raise EntityNotFoundError(
                f"{self._entity_name} с id={entity_id} не найден(а)."
            )
        if int(getattr(entity, "id")) != entity_id:
            raise ValueError("Идентификатор объекта нельзя изменить при обновлении.")
        self._items[entity_id] = entity
        return entity

    def delete(self, entity_id: int) -> T:
        try:
            return self._items.pop(entity_id)
        except KeyError as exc:
            raise EntityNotFoundError(
                f"{self._entity_name} с id={entity_id} не найден(а)."
            ) from exc

    def clear(self) -> None:
        self._items.clear()

    def __len__(self) -> int:
        return len(self._items)
```

Файл service.py

```
"""Сервисный слой: CRUD и контроль связей предметной области."""


from __future__ import annotations


from dataclasses import replace
from typing import Any, TypeVar


from .exceptions import BusinessRuleError, DuplicateEntityError
from .models import (
    CinemaData,
    Device,
    FavoriteList,
    Genre,
    Movie,
    Payment,
    PromoCode,
    Review,
    Subscription,
    SubscriptionPlan,
    User,
    WatchHistoryEntry,
)
from .repository import InMemoryRepository


T = TypeVar("T")
MODEL_TYPES: tuple[type[Any], ...] = (
    User,
    Genre,
    Movie,
    SubscriptionPlan,
    Subscription,
    Review,
    WatchHistoryEntry,
    FavoriteList,
    Payment,
    PromoCode,
    Device,
)



class CinemaService:
    """Предоставляет операции управления данными онлайн-кинотеатра."""


    def __init__(self) -> None:
        self._repositories: dict[type[Any], InMemoryRepository[Any]] = {
            model_type: InMemoryRepository(model_type.__name__)
            for model_type in MODEL_TYPES
        }


    def create(self, entity: T) -> T:
        """Добавляет объект после проверки уникальности и внешних связей."""
        model_type = type(entity)
        repository = self._repository_for(model_type)
        self._check_unique(entity)
        self._validate_relations(entity)
        return repository.add(entity)


    def get(self, model_type: type[T], entity_id: int) -> T:
        """Возвращает объект по идентификатору."""
        return self._repository_for(model_type).get(entity_id)


    def get_all(self, model_type: type[T]) -> list[T]:
        """Возвращает все объекты заданного типа."""
        return self._repository_for(model_type).get_all()


    def update(self, model_type: type[T], entity_id: int, **changes: Any) -> T:
        """Обновляет поля объекта, сохраняя его идентификатор."""
        if "id" in changes:
            raise BusinessRuleError("Изменение идентификатора запрещено.")
        repository = self._repository_for(model_type)
```

```
    current = repository.get(entity_id)
    updated = replace(current, **changes)
    self._check_unique(updated, exclude_id=entity_id)
    self._validate_relations(updated)
    return repository.update(entity_id, updated)


def delete(self, model_type: type[T], entity_id: int) -> T:
    """Удаляет объект, если на него не ссылаются другие сущности."""
    repository = self._repository_for(model_type)
    entity = repository.get(entity_id)
    self._check_delete_restrictions(entity)
    return repository.delete(entity_id)


def search_movies(self, query: str) -> list[Movie]:
    """Ищет фильмы по названию, описанию или имени режиссера."""
    normalized_query = query.strip().casefold()
    if not normalized_query:
        return self.get_all(Movie)
    return [
        movie
        for movie in self.get_all(Movie)
        if normalized_query in movie.title.casefold()
        or normalized_query in movie.description.casefold()
        or normalized_query in movie.director.casefold()
    ]


def export_data(self) -> CinemaData:
    """Собирает текущее состояние репозиториев в единый объект данных."""
    return CinemaData(
        users=self.get_all(User),
        genres=self.get_all(Genre),
        movies=self.get_all(Movie),
        plans=self.get_all(SubscriptionPlan),
        subscriptions=self.get_all(Subscription),
        reviews=self.get_all(Review),
        watch_history=self.get_all(WatchHistoryEntry),
        favorite_lists=self.get_all(FavoriteList),
        payments=self.get_all(Payment),
        promo_codes=self.get_all(PromoCode),
        devices=self.get_all(Device),
    )


def import_data(self, data: CinemaData) -> None:
    """Заменяет текущее состояние данными с повторной проверкой связей."""
    replacement = CinemaService()
    # Порядок важен: сначала создаются объекты, на которые ссылаются другие.
    ordered_collections: tuple[tuple[type[Any], list[Any]], ...] = (
        (User, data.users),
        (Genre, data.genres),
        (Movie, data.movies),
        (SubscriptionPlan, data.plans),
        (Subscription, data.subscriptions),
        (Review, data.reviews),
        (WatchHistoryEntry, data.watch_history),
        (FavoriteList, data.favorite_lists),
        (Payment, data.payments),
        (PromoCode, data.promo_codes),
        (Device, data.devices),
    )
    for _, entities in ordered_collections:
        for entity in entities:
            replacement.create(entity)
    self._repositories = replacement._repositories


def _repository_for(self, model_type: type[T]) -> InMemoryRepository[T]:
    if model_type not in self._repositories:
        raise TypeError(f"Тип {model_type.__name__} не поддерживается сервисом.")
    return self._repositories[model_type]


def _check_unique(self, entity: Any, exclude_id: int | None = None) -> None:
    """Проверяет уникальные бизнес-ключи отдельных сущностей."""
    if isinstance(entity, User):
        duplicates = [
            item for item in self.get_all(User)
            if item.email == entity.email and item.id != exclude_id
        ]
```

```
            if duplicates:
                raise DuplicateEntityError(
                    "Пользователь с такой почтой уже существует."
                )
        elif isinstance(entity, Genre):
            duplicates = [
                item for item in self.get_all(Genre)
                if (
                    item.name.casefold() == entity.name.casefold()
                    and item.id != exclude_id
                )
            ]
            if duplicates:
                raise DuplicateEntityError("Жанр с таким названием уже существует.")
        elif isinstance(entity, PromoCode):
            duplicates = [
                item for item in self.get_all(PromoCode)
                if item.code == entity.code and item.id != exclude_id
            ]
            if duplicates:
                raise DuplicateEntityError("Такой промокод уже существует.")


    def _validate_relations(self, entity: Any) -> None:
        """Проверяет внешние ключи и допустимость связей между объектами."""
        if isinstance(entity, Movie):
            for genre_id in entity.genre_ids:
                self.get(Genre, genre_id)
        elif isinstance(entity, Subscription):
            self.get(User, entity.user_id)
            self.get(SubscriptionPlan, entity.plan_id)
            # Изменение владельца не должно делать уже созданные платежи чужими.
            if any(
                payment.subscription_id == entity.id
                and payment.user_id != entity.user_id
                for payment in self.get_all(Payment)
            ):
                raise BusinessRuleError(
                    "Нельзя сменить владельца подписки: к ней привязан платеж "
                    "другого пользователя."
                )
        elif isinstance(entity, Review):
            self.get(User, entity.user_id)
            self.get(Movie, entity.movie_id)
            if any(
                item.user_id == entity.user_id and item.movie_id == entity.movie_id
                and item.id != entity.id
                for item in self.get_all(Review)
            ):
                raise BusinessRuleError("Пользователь уже оставил отзыв на этот фильм.")
        elif isinstance(entity, WatchHistoryEntry):
            self.get(User, entity.user_id)
            self.get(Movie, entity.movie_id)
        elif isinstance(entity, FavoriteList):
            self.get(User, entity.user_id)
            for movie_id in entity.movie_ids:
                self.get(Movie, movie_id)
        elif isinstance(entity, Payment):
            self.get(User, entity.user_id)
            subscription = self.get(Subscription, entity.subscription_id)
            if subscription.user_id != entity.user_id:
                raise BusinessRuleError(
                    "Платеж и подписка должны принадлежать одному пользователю."
                )
        elif isinstance(entity, Device):
            self.get(User, entity.user_id)


    def _check_delete_restrictions(self, entity: Any) -> None:
        """Не позволяет удалить объект, на который существуют ссылки."""
        references: list[str] = []
        if isinstance(entity, Genre):
            if any(entity.id in item.genre_ids for item in self.get_all(Movie)):
                references.append("фильмы")
        elif isinstance(entity, Movie):
            if any(item.movie_id == entity.id for item in self.get_all(Review)):
                references.append("отзывы")
            if any(
                item.movie_id == entity.id
                for item in self.get_all(WatchHistoryEntry)
            ):
```

```
            references.append("история просмотров")
        if any(entity.id in item.movie_ids for item in self.get_all(FavoriteList)):
            references.append("списки избранного")
    elif isinstance(entity, User):
        linked_types = (
            Subscription, Review, WatchHistoryEntry, FavoriteList, Payment, Device
        )
        for linked_type in linked_types:
            if any(item.user_id == entity.id for item in self.get_all(linked_type)):
                references.append(linked_type.__name__)
    elif isinstance(entity, SubscriptionPlan):
        if any(item.plan_id == entity.id for item in self.get_all(Subscription)):
            references.append("подписки")
    elif isinstance(entity, Subscription):
        if any(item.subscription_id == entity.id for item in self.get_all(Payment)):
            references.append("платежи")


    if references:
        joined = ", ".join(references)
        raise BusinessRuleError(
            f"Нельзя удалить {type(entity).__name__} id={entity.id}: "
            f"объект используется в данных ({joined})."
        )
```

Файл storage.py

```
"""Сериализация и десериализация набора данных в JSON и XML."""


from __future__ import annotations


from abc import ABC, abstractmethod
import json
from pathlib import Path
import xml.etree.ElementTree as ET
from typing import Any


from .exceptions import StorageError
from .models import CinemaData



class StorageAdapter(ABC):
    """Общий интерфейс файлового хранилища."""


    @abstractmethod
    def save(self, data: CinemaData, path: str | Path) -> None:
        """Сохраняет набор данных в файл."""


    @abstractmethod
    def load(self, path: str | Path) -> CinemaData:
        """Загружает набор данных из файла."""



class JsonStorage(StorageAdapter):
    """Файловое хранилище формата JSON."""


    def save(self, data: CinemaData, path: str | Path) -> None:
        target = Path(path)
        try:
            target.parent.mkdir(parents=True, exist_ok=True)
            target.write_text(
                json.dumps(data.to_dict(), ensure_ascii=False, indent=4),
                encoding="utf-8",
            )
        except (OSError, TypeError, ValueError) as exc:
            raise StorageError(
                f"Не удалось сохранить JSON-файл {target}: {exc}"
            ) from exc


    def load(self, path: str | Path) -> CinemaData:
        target = Path(path)
        try:
            payload = json.loads(target.read_text(encoding="utf-8"))
            if not isinstance(payload, dict):
                raise ValueError("Корневой элемент JSON должен быть объектом.")
            return CinemaData.from_dict(payload)
        except (OSError, json.JSONDecodeError, TypeError, ValueError, KeyError) as exc:
            raise StorageError(
                f"Не удалось загрузить JSON-файл {target}: {exc}"
            ) from exc



class XmlStorage(StorageAdapter):
    """Файловое хранилище формата XML на базе ElementTree."""


    def save(self, data: CinemaData, path: str | Path) -> None:
        target = Path(path)
        try:
            target.parent.mkdir(parents=True, exist_ok=True)
            root = ET.Element("online_cinema", attrib={"schema_version": "1.0"})
            for collection_name, records in data.to_dict().items():
                collection = ET.SubElement(root, collection_name)
                for record in records:
                    self._append_value(collection, "item", record)
```

```
        tree = ET.ElementTree(root)
        ET.indent(tree, space="    ")
        tree.write(target, encoding="utf-8", xml_declaration=True)
    except (OSError, TypeError, ValueError, ET.ParseError) as exc:
        raise StorageError(
            f"Не удалось сохранить XML-файл {target}: {exc}"
        ) from exc


def load(self, path: str | Path) -> CinemaData:
    target = Path(path)
    try:
        root = ET.parse(target).getroot()
        if root.tag != "online_cinema":
            raise ValueError(
                "Корневой элемент XML должен называться online_cinema."
            )
        payload: dict[str, Any] = {}
        for collection in root:
            payload[collection.tag] = [
                self._element_to_value(item) for item in collection.findall("item")
            ]
        return CinemaData.from_dict(payload)
    except (OSError, ET.ParseError, TypeError, ValueError, KeyError) as exc:
        raise StorageError(
            f"Не удалось загрузить XML-файл {target}: {exc}"
        ) from exc


@classmethod
def _append_value(cls, parent: ET.Element, tag: str, value: Any) -> None:
    """Рекурсивно добавляет словари и списки в XML-дерево."""
    element = ET.SubElement(parent, tag)
    if isinstance(value, dict):
        for key, nested_value in value.items():
            cls._append_value(element, key, nested_value)
    elif isinstance(value, list):
        for nested_value in value:
            cls._append_value(element, "item", nested_value)
    elif value is not None:
        element.text = str(value)


@classmethod
def _element_to_value(cls, element: ET.Element) -> Any:
    """Преобразует XML-элемент в словарь, список или строковое значение."""
    children = list(element)
    if not children:
        return element.text or ""
    if all(child.tag == "item" for child in children):
        return [cls._element_to_value(child) for child in children]
    return {child.tag: cls._element_to_value(child) for child in children}
```

Файл sample_data.py

```
"""Демонстрационный набор данных для проверки приложения."""


from .models import (
    CinemaData,
    Device,
    FavoriteList,
    Genre,
    Movie,
    Payment,
    PaymentStatus,
    PromoCode,
    Review,
    Subscription,
    SubscriptionPlan,
    SubscriptionStatus,
    User,
    WatchHistoryEntry,
)
from .service import CinemaService



def build_sample_data() -> CinemaData:
    """Создает согласованный демонстрационный набор данных."""
    return CinemaData(
        users=[
            User(1, "Иван Петров", "ivan@example.com"),
            User(2, "Елена Смирнова", "elena@example.com"),
        ],
        genres=[
            Genre(1, "Фантастика", "Научная фантастика и космические истории."),
            Genre(2, "Триллер", "Напряженные истории и расследования."),
            Genre(3, "Драма", "Истории о людях и важных решениях."),
        ],
        movies=[
            Movie(
                1,
                "Последний сигнал",
                "Инженер получает сообщение, которое было отправлено из будущего.",
                2024,
                118,
                "16+",
                [1, 2],
                "А. Воронов",
                ["М. Орлова", "И. Соколов"],
            ),
            Movie(
                2,
                "Город без сна",
                "Журналист расследует серию загадочных событий в большом городе.",
                2023,
                104,
                "16+",
                [2, 3],
                "Е. Миронова",
                ["А. Лебедев", "Н. Ким"],
            ),
            Movie(
                3,
                "Точка возврата",
                "Путешественник пытается восстановить отношения с семьей.",
                2022,
                96,
                "12+",
                [3],
                "Д. Волков",
                ["С. Белова", "П. Назаров"],
            ),
        ],
        plans=[
            SubscriptionPlan(1, "Стандарт", 299.0, 30, 2, "Full HD"),
            SubscriptionPlan(2, "Премиум", 499.0, 30, 5, "4K"),
        ],
        subscriptions=[
            Subscription(
                1, 1, 1, "2026-10-01", "2026-11-01", SubscriptionStatus.ACTIVE
            )
```

```
        ],
        reviews=[
            Review(
                1, 1, 1, 9, "Интересный сюжет и хорошая динамика.",
                "2026-10-10T18:30:00"
            )
        ],
        watch_history=[
            WatchHistoryEntry(1, 1, 1, 3540, "2026-10-29T21:15:00"),
            WatchHistoryEntry(2, 1, 3, 1200, "2026-10-25T20:05:00"),
        ],
        favorite_lists=[FavoriteList(1, 1, "Посмотреть позже", [2, 3])],
        payments=[
            Payment(
                1, 1, 1, 299.0, "Банковская карта", PaymentStatus.PAID,
                "2026-10-01T09:00:00"
            )
        ],
        promo_codes=[PromoCode(1, "START20", 20, "2026-12-31", True)],
        devices=[Device(1, 1, "Домашний телевизор", "smart_tv", "2026-10-29T21:00:00")],
    )



def create_sample_service() -> CinemaService:
    """Создает сервис и заполняет его демонстрационными данными."""
    service = CinemaService()
    service.import_data(build_sample_data())
    return service
```

Файл cli.py

```
"""Текстовый интерфейс приложения онлайн-кинотеатра."""

from pathlib import Path

from .exceptions import CinemaError
from .models import Genre, Movie
from .sample_data import create_sample_service
from .service import CinemaService
from .storage import JsonStorage, XmlStorage



DATA_DIR = Path(__file__).resolve().parents[1] / "data"
JSON_PATH = DATA_DIR / "online_cinema.json"
XML_PATH = DATA_DIR / "online_cinema.xml"



def _print_movies(service: CinemaService) -> None:
    movies = service.get_all(Movie)
    if not movies:
        print("Каталог пуст.")
        return
    for movie in movies:
        print(
            f"[{movie.id}] {movie.title} ({movie.release_year}), "
            f"{movie.duration_minutes} мин., {movie.age_rating}"
        )



def main() -> None:
    """Запускает меню и перехватывает ожидаемые ошибки пользовательского ввода."""
    service = create_sample_service()
    json_storage = JsonStorage()
    xml_storage = XmlStorage()

    while True:
        print("\n=== LUMEN | Онлайн-кинотеатр ===")
        print("1. Показать каталог")
        print("2. Найти фильм")
        print("3. Добавить фильм")
        print("4. Изменить описание фильма")
        print("5. Удалить фильм")
        print("6. Сохранить данные в JSON")
        print("7. Сохранить данные в XML")
        print("8. Загрузить данные из JSON")
        print("9. Загрузить данные из XML")
        print("0. Выход")

        try:
            choice = input("Выберите действие: ").strip()
            if choice == "1":
                _print_movies(service)
            elif choice == "2":
                query = input("Название, описание или режиссер: ")
                for movie in service.search_movies(query):
                    print(f"[{movie.id}] {movie.title} ({movie.release_year})")
            elif choice == "3":
                movie = Movie(
                    id=int(input("ID фильма: ")),
                    title=input("Название: ").strip(),
                    description=input("Описание: ").strip(),
                    release_year=int(input("Год выпуска: ")),
                    duration_minutes=int(input("Длительность в минутах: ")),
                    age_rating=input(
                        "Возрастной рейтинг (0+/6+/12+/16+/18+): "
                    ).strip(),
                    genre_ids=[
                        int(value)
                        for value in input("ID жанров через запятую: ").split(",")
                        if value.strip()
                    ],
```

```
                    director=input("Режиссер: ").strip() or "Не указан",
                )
                service.create(movie)
                print("Фильм добавлен.")
            elif choice == "4":
                movie_id = int(input("ID фильма: "))
                description = input("Новое описание: ").strip()
                service.update(Movie, movie_id, description=description)
                print("Описание обновлено.")
            elif choice == "5":
                movie_id = int(input("ID фильма: "))
                service.delete(Movie, movie_id)
                print("Фильм удален.")
            elif choice == "6":
                json_storage.save(service.export_data(), JSON_PATH)
                print(f"Данные сохранены: {JSON_PATH}")
            elif choice == "7":
                xml_storage.save(service.export_data(), XML_PATH)
                print(f"Данные сохранены: {XML_PATH}")
            elif choice == "8":
                service.import_data(json_storage.load(JSON_PATH))
                print("Данные загружены из JSON.")
            elif choice == "9":
                service.import_data(xml_storage.load(XML_PATH))
                print("Данные загружены из XML.")
            elif choice == "0":
                print("Работа завершена.")
                break
            else:
                print("Неизвестная команда. Выберите пункт от 0 до 9.")
        except (CinemaError, ValueError, TypeError) as exc:
            # Ошибка выводится пользователю, но не завершает главный цикл приложения.
            print(f"Ошибка: {exc}")
        except (EOFError, KeyboardInterrupt):
            print("\nРабота завершена пользователем.")
            break
```

Файл __init__.py

```
"""Учебное приложение «Онлайн-кинотеатр» для лабораторной работы № 1."""

from .service import CinemaService

__all__ = ["CinemaService"]
```

Файл __main__.py

```
from .cli import main

if __name__ == "__main__":
    main()
```